23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No :	**D9**
Team Members :	
Roll No	**Student Name**
AM.SC.U4CSE25359	ABEL BINU VARGHESE
AM.SC.U4CSE25360	ALIEN LAKSHMI C
AM.SC.U4CSE25366	KASINATH V
AM.SC.U4CSE25368	DEVANANDA JA
Project Title :	**Contact Management System**

CONTENTS

Sl. No.	**Chapter**
1	Abstract
2	Introduction
3	Requirements (Functional and Non-functional)
4	UI Design
5	Database Design
6	Database Selection and Justification
7	Implementation
8	Testing and Validation
9	Results and Screenshots
10	Challenges Faced and Solutions
11	Conclusion and Future Enhancements
	References

1. Abstract

The Contact Management System is a web-based database application developed to store and manage contact information in an organized manner. It provides a user interface for viewing, searching, adding, editing, deleting, categorizing, and marking contacts as favourites. PostgreSQL is used as the relational database, while a Node.js and Express backend exposes REST API endpoints to communicate with the database. The frontend is hosted using GitHub Pages and the backend is deployed on Render. The database design uses primary keys, foreign keys, uniqueness constraints, and cascading deletion to maintain data integrity. The system also includes tables intended for user accounts, contact groups, communication logs, and activity logs. The project demonstrates relational modelling, SQL table creation, CRUD operations, joins, API integration, and cloud deployment. The current implementation connects the contact operations to PostgreSQL; authentication and user/activity logging remain frontend-side and can be extended in future versions.

2. Introduction

2.1 Background

Contact information is often scattered across phone address books, spreadsheets, email applications, and handwritten records. When contact details are maintained in separate places, it becomes difficult to search for a person, update outdated information, group contacts, and avoid duplicate records. A database-driven contact management application provides a centralized way to organize this information.

2.2 Existing scenario and limitations

A basic address book or spreadsheet can store names and phone numbers, but it may provide limited validation, relationships between records, structured querying, and reliable multi-user data management. Duplicate phone numbers or email addresses can also be entered unless uniqueness is enforced. Browser-only storage is tied to a particular browser and device and is not a substitute for a shared server-side database.

2.3 Proposed system

The proposed Contact Management System combines a browser-based interface with a PostgreSQL database accessed through an Express REST API. The frontend sends HTTP requests to the backend for contact retrieval and CRUD operations. The backend validates incoming data and executes parameterized SQL statements. Contact categories are represented using groups and a junction table, while the schema also includes users and logging tables for future or supporting functionality.

2.4 Objectives

- Maintain contact details such as name, phone, email, address, birthday, and company.
- Support adding, viewing, editing, deleting, and searching contacts.
- Organize contacts into categories/groups and mark contacts as favourites.
- Use relational constraints to reduce duplicate data and preserve referential integrity.
- Deploy the frontend and backend so the application can be accessed through the web.
- Demonstrate SQL DDL, CRUD statements, joins, constraints, and database connectivity.

3. Requirements (Functional and Non-functional)

3.1 Functional requirements

- Display the list of contacts retrieved from the PostgreSQL database.
- Insert a contact with required name, phone, and email fields.
- Update an existing contact's details and category.
- Delete a contact and remove its dependent contact-group and communication-log records through cascading foreign keys.
- Mark or unmark a contact as a favourite.
- Search and browse contacts through the frontend interface.
- Prevent duplicate phone numbers and email addresses using validation and database uniqueness constraints.
- Provide a sign-in interface and local user management in the current frontend; server-side authentication is a future enhancement.
- Maintain group, communication-log, and activity-log tables in the database schema.

3.2 Non-functional requirements

Requirement	Description
Usability	The web interface provides a visual contact list, search field, category navigation, favourite filter, and forms for contact entry.
Performance	Indexed contact name and phone fields support common lookups; API requests should return only the data needed by the interface.
Integrity	Primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and ON DELETE CASCADE constraints protect the data.
Reliability	The backend uses PostgreSQL transactions for multi-step contact insert/update operations.
Security	SQL statements use parameters. Production use still requires server-side authentication, authorization, HTTPS, and secure credential handling.
Maintainability	Frontend, backend, and database schema are separated into distinct components.
Availability	Frontend is hosted on GitHub Pages and backend on Render; free hosting may sleep after inactivity and the free database has an expiry limitation.

4. UI Design

The user interface uses a dark theme with blue accents. The frontend is a static web application built with HTML, CSS, and JavaScript and is hosted on GitHub Pages. The backend is hosted separately and is accessed through HTTP requests.

4.1 Login screen

The login screen displays the Contact Management System title, Sign In and Sign Up tabs, username and password fields, and a Log In button. The current login/session logic is implemented in frontend browser storage; credentials are not yet verified by a backend authentication API.

4.2 Contact dashboard

The dashboard displays the contact list and summary cards for total contacts, categories, companies, and favourites. A side navigation area contains a search box, category list, alphabetical navigation, Manage Users, Activity Log, and the current user's profile controls.

4.3 Contact entry and editing

The Add Contact form accepts name, phone, email, address, birthday, company, and category. The backend requires name, phone, and email and rejects duplicate phone numbers or email addresses. The same form supports editing existing contact details.

4.4 Search, category and favourite controls

The dashboard provides search and category navigation, sorting, and a favourites-only filter. The favourite action is connected to the backend endpoint that toggles the is_favourite field.

4.5 Navigation flow

User opens the GitHub Pages site → signs in through the frontend → dashboard initializes → frontend requests GET /contacts from the Render API → user can add, edit, delete, or favourite a contact → backend updates PostgreSQL → frontend refreshes the displayed data.

Screenshot placeholders to insert: (1) Login screen, (2) Contact dashboard, (3) Add/Edit Contact form, (4) Empty or populated contact list, (5) pgAdmin contacts table, and (6) successful SQL execution output.

5. Database Design

5.1 Entities and attributes

Entity / Table	Attributes and keys
users	user_id (PK), username (UNIQUE, NOT NULL), password_hash (NOT NULL), role (CHECK constraint), created_at
contacts	contact_id (PK), name (NOT NULL), phone (UNIQUE), email (UNIQUE), address, birthday, company, is_favourite, user_id (FK → users.user_id)
groups	group_id (PK), group_name (UNIQUE, NOT NULL)
contact_group	contact_id (FK → contacts.contact_id), group_id (FK → groups.group_id), composite PK (contact_id, group_id)
communication_log	contact_id (FK → contacts.contact_id), log_id, type (NOT NULL), log_datetime, notes, logged_by (FK → users.user_id), composite PK (contact_id, log_id)
activity_log	activity_id (PK), user_id (FK → users.user_id), action_type (NOT NULL), description, timestamp

5.2 Relationships and cardinalities

- Users to contacts: one user may be associated with many contacts; each contact can reference zero or one user because contacts.user_id is nullable.
- Contacts to groups: many-to-many relationship implemented through contact_group. A contact may belong to multiple groups and a group may contain multiple contacts.
- Contacts to communication_log: one contact may have many communication-log entries. Deleting a contact cascades to its communication logs.
- Users to communication_log: one user may log many communications through logged_by.
- Users to activity_log: one user may have many activity-log records.

5.3 Text ER model

USERS (1) ———— (N) CONTACTS

CONTACTS (1) ———— (N) CONTACT_GROUP (N) ———— (1) GROUPS

CONTACTS (1) ———— (N) COMMUNICATION_LOG

USERS (1) ———— (N) COMMUNICATION_LOG through logged_by

USERS (1) ———— (N) ACTIVITY_LOG

The lines represent logical relationships; the actual optionality depends on nullable foreign-key columns.

5.4 Relational schema

- USERS(user_id PK, username UNIQUE, password_hash, role, created_at)
- CONTACTS(contact_id PK, name, phone UNIQUE, email UNIQUE, address, birthday, company, is_favourite, user_id FK)
- GROUPS(group_id PK, group_name UNIQUE)
- CONTACT_GROUP(contact_id PK/FK, group_id PK/FK)
- COMMUNICATION_LOG(contact_id PK/FK, log_id PK, type, log_datetime, notes, logged_by FK)
- ACTIVITY_LOG(activity_id PK, user_id FK, action_type, description, timestamp)

5.5 ER-to-relational conversion

Each strong entity (users, contacts, groups, and activity_log) is mapped to its own relation with a primary key. The many-to-many relationship between contacts and groups is converted into contact_group, whose composite primary key prevents the same contact-group pair from being stored more than once. The one-to-many communication-log relationship is represented by a foreign key in communication_log. Foreign keys implement referential integrity.

5.6 Functional dependencies and normalization

Examples of functional dependencies include user_id → username, password_hash, role, created_at; contact_id → name, phone, email, address, birthday, company, is_favourite, user_id; group_id → group_name; and activity_id → user_id, action_type, description, timestamp. The unique constraints on username, phone, email, and group_name enforce additional candidate-key properties.

The schema is designed to be in at least Third Normal Form for the represented attributes: attributes are stored with the entity they describe, group names are separated from contacts, and the many-to-many association is stored in a junction table. The design reduces update, insertion, and deletion anomalies. A more formal normalization proof should be added if required by the course rubric.

5.7 Constraints and indexes

- Primary keys uniquely identify rows.
- UNIQUE constraints prevent duplicate usernames, contact phone numbers, emails, and group names.
- NOT NULL constraints require essential attributes.
- The users.role CHECK constraint permits Administrator, Editor, or Viewer.
- Foreign keys enforce references between related tables.
- ON DELETE CASCADE removes dependent contact_group and communication_log rows when a contact is deleted.
- Indexes idx_contacts_name and idx_contacts_phone support common name and phone lookups.

6. Database Selection and Justification

PostgreSQL was selected as the relational database management system. It is open-source, supports standard SQL, foreign keys, transactions, constraints, indexes, and multi-table joins. These features suit a contact management application in which contact records must remain consistent while categories and logs are linked through relationships. PostgreSQL also integrates with Node.js through the pg library and can be deployed as a managed database on Render. pgAdmin provides a graphical interface for creating tables, executing SQL

statements, and inspecting query results. The project uses a free Render database for demonstration; its displayed expiry date and service limitations should be considered for long-term availability.

7. Implementation

7.1 Technology stack

Component	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User interface, forms, search, category navigation, display and client-side session state
Backend	Node.js, Express.js	REST API endpoints and request handling
Database	PostgreSQL	Persistent relational storage
Database client	pg (node-postgres)	Connection pool and parameterized SQL queries
Database GUI	pgAdmin 4	Database registration, schema creation, SQL execution and table inspection
Frontend hosting	GitHub Pages	Public hosting of static frontend
Backend/database hosting	Render	Deployment of Express service and PostgreSQL database

7.2 Database creation and table creation

The database database_management_db was created on Render. The schema was created by running SQL through pgAdmin after registering a connection to the Render database using its external hostname, port 5432, database name, username, and password. The SQL script created six tables and two indexes.

7.3 Important SQL snippets

Example: contacts table

```
CREATE TABLE contacts (
    contact_id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    phone VARCHAR(20) UNIQUE,
    email VARCHAR(150) UNIQUE,
    address TEXT,
    birthday DATE,
    company VARCHAR(150),
    is_favourite BOOLEAN DEFAULT FALSE,
    user_id INTEGER REFERENCES users(user_id)
);
```

Example: retrieve contacts with their group/category

```
SELECT c.contact_id AS id, c.name, c.phone, c.email,
       c.address, c.birthday, c.company,
       c.is_favourite AS favourite,
       COALESCE(g.group_name, 'Personal') AS category
FROM contacts c
LEFT JOIN LATERAL (
    SELECT gr.group_name
    FROM contact_group cg
    JOIN groups gr ON gr.group_id = cg.group_id
    WHERE cg.contact_id = c.contact_id
    ORDER BY gr.group_id
    LIMIT 1
) g ON TRUE
ORDER BY c.name;
```

7.4 REST API endpoints

Method	Endpoint	Purpose

GET	/contacts	Retrieve all contacts with a category value
POST	/contacts	Validate and insert a contact and its group association
PUT	/contacts/:id	Update a contact and replace its group association
PATCH	/contacts/:id/favourite	Toggle the favourite flag
DELETE	/contacts/:id	Delete a contact
GET	/	Return a simple backend health message

7.5 CRUD and transaction handling

The backend uses parameterized SQL queries (for example, placeholders such as $1 and $2) rather than concatenating user input into SQL. The POST and PUT handlers use transactions to update the contact and its group association together. A transaction is committed only when the related operations succeed; errors cause a rollback. Duplicate phone numbers and email addresses are checked before insertion or update, and database uniqueness constraints provide an additional safeguard.

7.6 Deployment and configuration

The frontend is hosted at https://alien2088.github.io/23CSE202_ContactManagementSystem_D9/ and the backend at https://two3cse202-contactmanagementsystem-d9.onrender.com. The backend is configured to use DATABASE_URL when deployed and local DB environment variables for local development. The .env file and node_modules are excluded from GitHub. The deployed backend was verified by opening the root route and testing GET /contacts, which returned an empty JSON array after the tables were created.

7.7 Scope note

The database schema contains user and logging tables, but the current frontend login and user/activity-log behaviour still relies on localStorage and the backend does not yet provide authentication endpoints. The current database-backed scope is contact retrieval and contact CRUD/favourite operations. Server-side authentication and authorization should be implemented before using the application for real personal data.

8. Testing and Validation

The following test cases reflect the implementation and checks performed during development. Replace or supplement the actual-result column with final test evidence captured from the completed application.

ID	Test / Input	Expected output	Actual result / Status
TC-01	Open backend root URL	Health message indicates backend is running	Observed: backend health message displayed — PASS
TC-02	GET /contacts after schema creation	JSON array is returned; empty array when no records exist	Observed: [] returned — PASS
TC-03	Insert contact with name, phone, email	Contact is inserted and returned with generated ID	Verify with final UI test — PENDING
TC-04	Insert without name, phone, or email	HTTP 400 validation error	Defined in API code — implementation check
TC-05	Insert duplicate phone or email	HTTP 409 duplicate error	Defined in API code and database constraints — implementation check
TC-06	Update existing contact	HTTP 200 JSON response and updated row	Verify with final UI test — PENDING
TC-07	Favourite toggle	Favourite flag changes and is returned	Verify with final UI test — PENDING
TC-08	Delete existing contact	Contact is deleted; dependent rows cascade	Verify with final UI test — PENDING
TC-09	Delete nonexistent contact	HTTP 404 response	Defined in API code — implementation check
TC-10	Refresh frontend after inserting contact	Contact remains because data is in PostgreSQL	Verify with final UI test — PENDING

8.1 Data integrity validation

Validation is provided at both application and database levels. The API checks required fields and duplicate phone/email values. PostgreSQL enforces primary keys, uniqueness, foreign-key relationships, role values, and

non-null requirements. Transaction rollback prevents a partially completed contact/group operation if an intermediate SQL statement fails.

8.2 Testing still to complete

- Confirm Add Contact through the live frontend and verify the new row in pgAdmin.
- Edit the test contact, refresh the website, and verify the updated values in PostgreSQL.
- Toggle favourite and verify contacts.is_favourite changes.
- Delete the test contact and verify that the row is removed.
- Capture screenshots of both successful operations and validation errors.

9. Results and Screenshots

During deployment, the Express service successfully started on Render and reported a successful PostgreSQL connection. Initially, GET /contacts returned an error because the contacts table had not been created in the Render database. After the schema SQL was executed against the Render database, GET /contacts returned an empty JSON array, confirming that the API could query the table. The frontend login screen and contact dashboard loaded successfully. At the time of preparing this report, the dashboard displayed zero contacts; final end-to-end CRUD results should be added after the test cases in Chapter 8 are completed.

Figure	Screenshot to insert	Caption / purpose
Fig. 1	Login page	Shows the sign-in interface and default demo credentials displayed by the frontend.
Fig. 2	Contact dashboard	Shows contact summary cards, search, categories, favourites filter, and navigation.
Fig. 3	pgAdmin database tree	Shows the Render Database connection and the public schema tables.
Fig. 4	SQL execution output	Shows successful execution of the CREATE TABLE and CREATE INDEX statements.
Fig. 5	Backend /contacts URL	Shows the API returning [] when the contacts table is empty.
Fig. 6	Contact added and pgAdmin row	Insert the final screenshot after confirming an end-to-end add operation.

Insert the actual screenshots in the middle column or directly below each figure entry before submission. Avoid including database passwords, complete connection URLs, or other secrets in screenshots.

10. Challenges Faced and Solutions

Challenge	Solution / learning
The deployed API initially returned “relation contacts does not exist”.	The tables had been created in the local database rather than the Render database. A pgAdmin connection was registered to the Render database and the schema SQL was executed there.
Connecting pgAdmin to the hosted database.	The external hostname, port 5432, database name, username, and password were entered as separate connection fields.
Keeping database credentials out of source control.	The backend .gitignore excludes .env and node_modules; Render uses the DATABASE_URL environment variable.
Frontend and backend were hosted separately.	The frontend API base URL was configured to use the public Render service URL and the backend enabled CORS.
Frontend used localStorage while the database was on PostgreSQL.	The contact-loading and CRUD functions were changed to call REST API endpoints. User/session and activity-log behaviour remains local and is documented as a limitation.
A JavaScript syntax error occurred during contact deletion changes.	The complete confirmPendingDelete() function was rewritten with balanced braces and async/await error handling.
Free hosting may sleep or expire.	The report notes that Render free services may spin down after inactivity and the free database has a stated expiry; long-term deployment requires an appropriate hosting plan or migration.

11. Conclusion and Future Enhancements

11.1 Conclusion

The Contact Management System demonstrates the design and deployment of a relational database application. A PostgreSQL schema was created with six related tables, primary and foreign keys, uniqueness and check constraints, cascading deletion, and indexes. A Node.js/Express API provides endpoints for retrieving, inserting, updating, deleting, and favouriting contacts. The frontend and backend were deployed separately using GitHub Pages and Render. Testing confirmed that the deployed backend connected to PostgreSQL and returned contact data after the missing tables were created. Final CRUD verification screenshots should be included before submission.

11.2 Future enhancements

- Implement server-side registration, login, password hashing, session/token handling, and role-based authorization.
- Move activity logs and communication logs from localStorage to their PostgreSQL tables.
- Support multiple categories per contact in the UI, matching the many-to-many database design.
- Add pagination and more advanced search/filter queries for larger contact lists.
- Add import/export, backup and restore, and report-generation features.
- Improve validation, API error messages, automated tests, and accessibility.
- Use environment-specific CORS rules, rotate exposed database credentials, and secure production configuration.
- Choose persistent hosting or a backup/migration strategy before the free database expiry date.

References

1. PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs/
2. Express.js. Express Documentation. https://expressjs.com/
3. node-postgres. node-postgres (pg) Documentation. https://node-postgres.com/
4. pgAdmin Development Team. pgAdmin 4 Documentation. https://www.pgadmin.org/docs/
5. GitHub Docs. GitHub Pages Documentation. https://docs.github.com/en/pages
6. Render. Render Documentation. https://render.com/docs
7. MDN Web Docs. Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API